

HANDS_ON_LabGS_ExecuteScript_Groovy

Lab GS: ExecuteScript mit Groovy (file-only)

Apache NiFi 2.8.0 · UI: <https://localhost:8443/nifi> · Login admin N=/Users/grigo/gits/seminars/it-schulungen.com/Apache_NiFi/nifi-2.8.0 Fertiges Referenz-Lab auf der Canvas: “**Lab GS - ExecuteScript Groovy (file)**” Live gebaut und verifiziert.

Lernziel: Eigene Logik per Skript in den Flow bringen, wenn kein Standard-Prozessor passt. ExecuteScript transformiert den **Inhalt** (Großbuchstaben) und setzt **Attribute** (Zeilenzahl, Marker). Zeigt das Grundmuster `session.get` → `write/putAttribute` → `transfer`.

Wichtig (NiFi 2.x): ExecuteScript unterstützt nur noch **Groovy** und **Clojure**. Jython/Python, JavaScript/Nashorn, JRuby, Lua sind raus. Python gibt es separat über die native Python-Processor-API, **nicht** über ExecuteScript.

Flow:

```
GenerateFlowFile → ExecuteScript (Groovy) → LogAttribute → PutFile  
(Testtext)          (UPPERCASE + Attribute)  (Log)          (Output)
```

Prozessoren

1. GenerateFlowFile (Testquelle)

- Custom Text =

```
hallo welt  
zeile zwei  
zeile drei
```

- Data Format = `Text`, Batch Size = `1`
- Scheduling → Run Schedule = `60` sec (sonst feuert es im Sekundentakt)

2. ExecuteScript (das Herzstück)

- **Script Engine** = `Groovy`
- **Script Body:**

```
import org.apache.nifi.processor.io.StreamCallback  
def ff = session.get()  
if (!ff) return  
def lineCount = 0  
ff = session.write(ff, { inS, outS ->  
    def text = inS.text  
    lineCount = text.readLines().size()  
    outS.write(text.toUpperCase().bytes)
```

```

} as StreamCallback)
ff = session.putAttribute(ff, 'groovy_verarbeitet', 'true')
ff = session.putAttribute(ff, 'zeilen', lineCount.toString())
session.transfer(ff, REL_SUCCESS)

```

- Relationship failure terminieren, success → LogAttribute

3. LogAttribute

- Log Level = info, Log FlowFile Properties = true
- zeigt u.a. groovy_verarbeitet und zeilen im nifi-app.log

4. PutFile

- Directory = \$N/data/output/labGroovy
- Create Missing Directories = true, Conflict Resolution = replace

Verdrahtung

Von	Relationship	Nach
GenerateFlowFile	success	ExecuteScript
ExecuteScript	success	LogAttribute
ExecuteScript	failure	terminieren
LogAttribute	success	PutFile
PutFile	success, failure	terminieren

Das ExecuteScript-Grundmuster (zum Verstehen)

Gebundene Objekte im Skript: - session : FlowFile holen/ändern/übergeben - context : Prozessor-Properties lesen (context.getProperty('x').getValue()) - log : ins nifi-app.log (log.info(...), log.error(...)) - REL_SUCCESS, REL_FAILURE : die Ausgangs-Relationships

Ablauf fast jedes Skripts:

```

def ff = session.get()           // FlowFile aus der Queue holen
if (!ff) return                  // nichts da -> beenden
// ... ff mit session.write / session.putAttribute aendern ...
session.transfer(ff, REL_SUCCESS) // in die Relationship geben

```

session.write(...) ersetzt den Inhalt, session.putAttribute(...) gibt eine **neue** FlowFile-Referenz zurück : darum jedes Mal ff = session... neu zuweisen.

Starten und prüfen

```

# PG starten (GenerateFlowFile feuert einmal sofort)
sleep 5
cat $N/data/output/labGroovy/*           # Inhalt in GROSSBUCHSTABEN

```

```
grep -A 20 LogAttribute $N/logs/nifi-app.log | grep -iE "groovy_verarbeitet|zeilen"  
# danach PG STOPPEN (sonst alle 60s neue Datei)
```

Verifiziertes Ergebnis (im Live-Test bestätigt)

Output-Datei-Inhalt:

```
HALLO WELT  
ZEILE ZWEI  
ZEILE DREI
```

Im LogAttribute-Block gesetzt: `groovy_verarbeitet = true`, `zeilen = 3`.

Weitere Groovy-Bausteine (zum Erweitern)

```
// JSON-Feld in Attribut  
import groovy.json.JsonSlurper  
def data = session.read(ff).withReader { new JsonSlurper().parse(it) }  
ff = session.putAttribute(ff, 'kundennr', data.kundennr.toString())  
  
// Eigenes Routing nach Inhalt  
def betrag = session.read(ff).text.readLines()[1].split(',')[2] as double  
session.transfer(ff, betrag >= 50 ? REL_SUCCESS : REL_FAILURE)  
  
// Property lesen + loggen  
def praefix = context.getProperty('praefix').getValue()  
log.info("Verarbeite ${ff.getAttribute('filename')}")
```

Reset

```
rm -f $N/data/output/labGroovy/*  
# PG stoppen
```

Häufige Stolpersteine

- **Script Engine fehlt/ungültig** → nur Groovy oder Clojure (2.x); kein Python/JS in ExecuteScript.
- **ff nicht neu zugewiesen** → `session.putAttribute/write` geben eine **neue** Referenz zurück; ohne `ff = ...` arbeitest du auf der alten und bekommst Fehler beim `transfer`.
- **Kein session.get() -Null-Check** → bei leerer Queue wirft das Skript; immer `if (!ff) return`.
- **GenerateFlowFile flutet** → Run Schedule hochsetzen (z.B. 60 sec) oder PG nach dem Test stoppen.
- `failure` **nicht terminiert/verbunden** → Prozessor invalid (△).